Programming with Python



7. Functions

- ✓ Introduction to Functions
- ✓ Mutable vs Immutable Arguments
- ✓ Types of function parameters
- ✓ Scope – Local vs Global
- ✓ Lambda Expressions
- ✓ Execution flow of funcions
- ✓ Examples

Function

"Do a task again and again…"

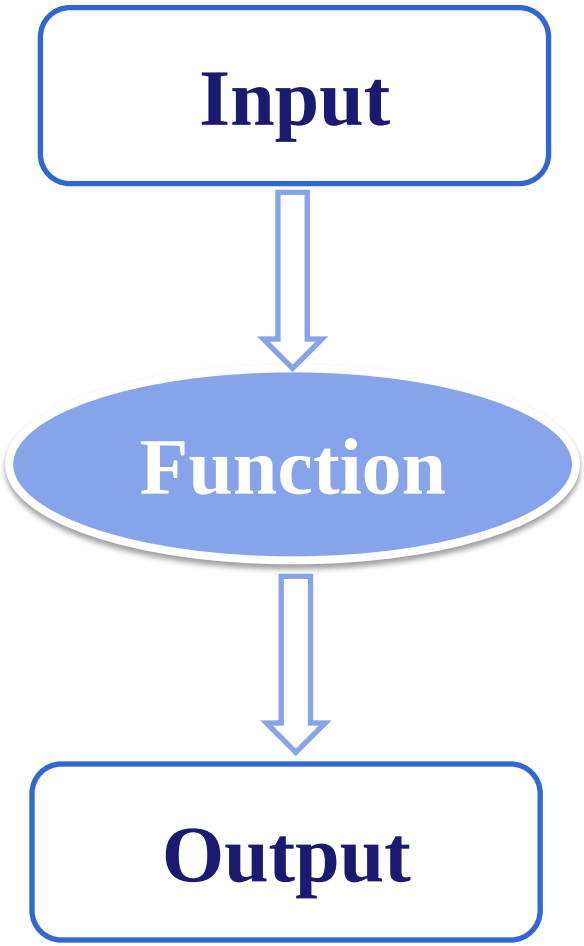
Search for a name in a list…

Sort the numbers…

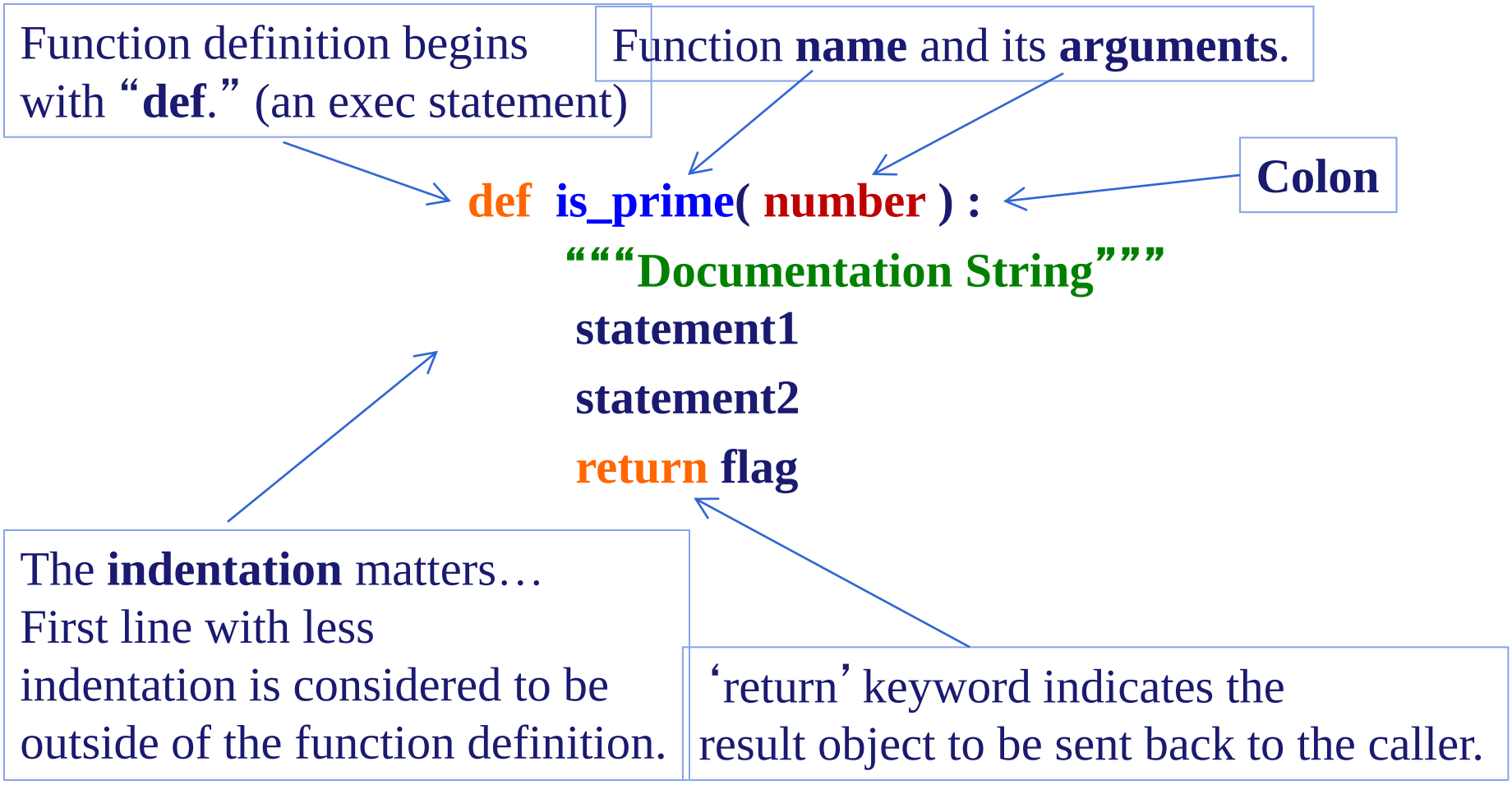
Check whether the given number is prime or not…

Function

- **Definition:** A function is a self-contained program segment that carries out some specific, well-defined task.
- Function will carry out its intended action whenever it is called.
- **Types:**
  - In-Built
  - User defined



Defining Functions



- **No header file or declaration of types of function or arguments**
- **Function definition can be nested in if or while/for statements**

Function

- "def" creates an object and assigns it to a name – function name
- Python "def" is an executable statement
- **"return"** can be kept anywhere in the function body
- We can keep def in if/for/while statements
- **Note:** We can assign the function name to any other variable/name

```
fun1 = fun  # It is like a object cloning
fun1( )
```

More about “def”

- “def” creates an object and assigns it to a name – function name
- Python “def” is an executable statement
- **“return”** can be kept anywhere in the function body
- We can keep def in if/for/while statements
- **Note:** We can assign the function name to any other variable/name

```
fun1 = fun  # It is like a object cloning
fun1( )
```

Calling a function

Syntax:

return_val = function_name(arguments)

Example:

```
# Function Definition
def sum(x, y):
    return x + y

# Function call
sum = sum(3, 4)
print(sum)

# Output
12
```

Pass by value or reference??

None of the above

In Python, every thing is an object…

Variables are just names assigned to objects, not place holders like C or C++ …

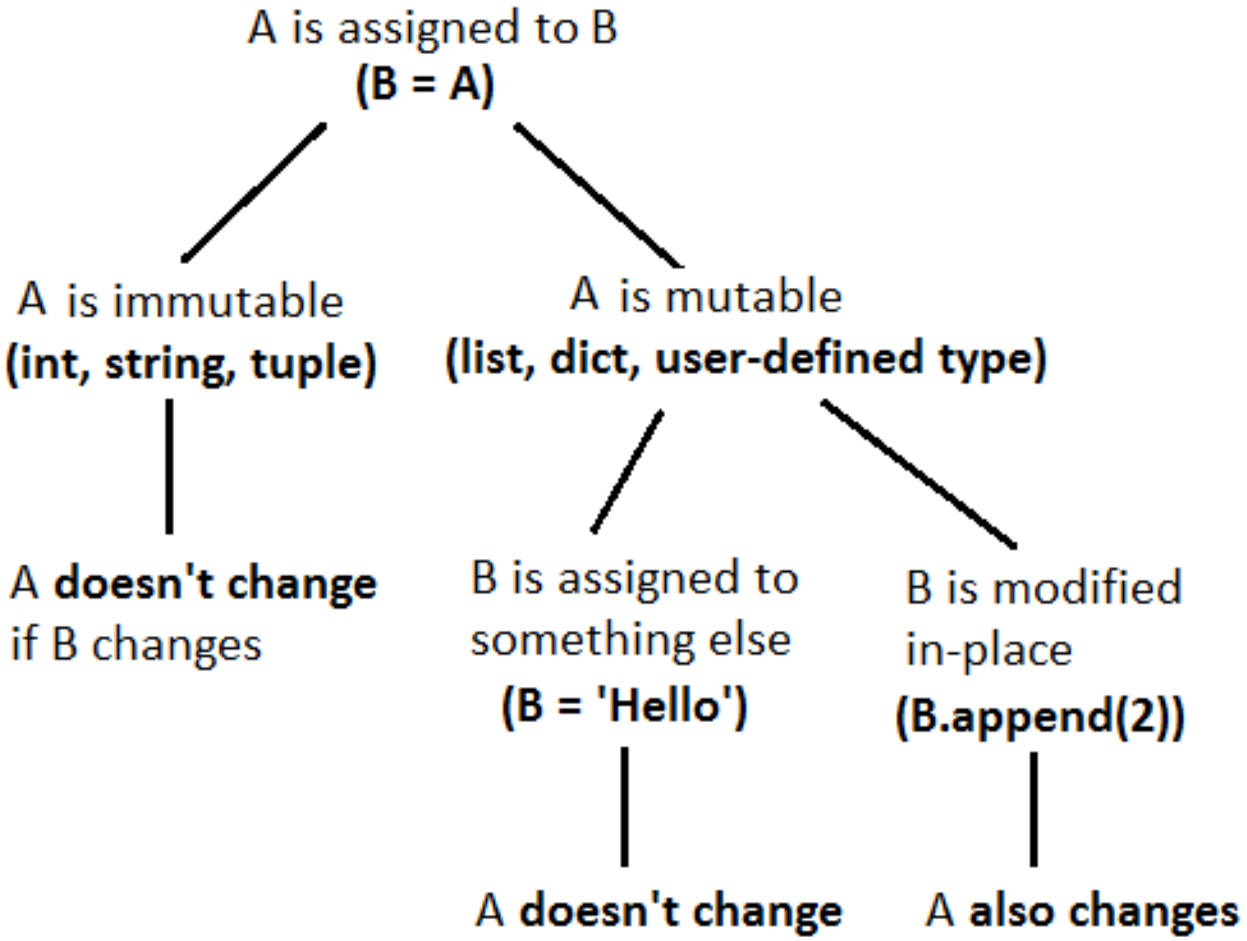
Python uses "Call by object" (OR)

"Call by object reference"

(OR) "Call by sharing" …

Pass by value or reference??

Understand how python treats variable assignments/Arguments



Passing immutable objects

Ex1: Passing Integers

```
# Function definition
>>> def swap(x, y):
        temp = x
        x = y
        y = temp
        return

# Call swap function
>>> a=5
>>> b=10
>>> swap(a,b)
>>> print(a,b)
5 10
```

Ex2: Passing Strings

```
# Function definition
>>> def edit(str2):
        str2 = "sam"

# call edit function
>>> str1 = "ram"
>>> edit(str1)
>>> print(str1)
'ram'
```

Passing mutable objects

Ex3: Passing Lists

```
# Function definition
>>> def append1(list2):
        list2[3] = 3

# Call append function
>>> list1 = [0, 1, 2]
>>> append1(list1)
>>> print(list1)
indexError: list
assignment is out of
range
```

We cannot go beyond the bounds.

Ex4: Passing Lists

```
# Function definition
>>> def append1(list2):
        list2[2] = 10

# Call append function
>>> list1 = [0, 1, 2]
>>> append1(list1)
>>> print(list1)
[0, 1, 10]
```

The change made to formal argument ‘list2’ will make change to the actual argument ‘list1’.

Types of Arguments

Types of Arguments

- Default Arguments
- Keyword Arguments
- Variable length Arguments/Arbitrary Arguments

Default Arguments

- Specify a default value for one or more arguments.
- A function that can be called with fewer arguments than it is defined to allow.
- **Example:**

default value of retries is 4

```
>>> def ask_ok(prompt, retries=4, complaint='Yes or no, please!'):
        while True:
            ok = input(prompt)
            if ok in ('y', 'ye', 'yes'):
                return True
            if ok in ('n', 'no', 'nop', 'nope'):
                return False
            retries = retries - 1
            if retries < 0:
                raise OSError('uncooperative user')
            print(complaint)
```

Default Arguments…

How to call the function “ask_ok”?

- giving only the mandatory argument:

```
ask_ok('Do you really want to quit?')
```

- giving one of the optional arguments:

```
ask_ok('OK to overwrite the file?', 2)
```

- giving all the arguments:

```
ask_ok('OK to overwrite the file?', 2, 'Come on, only yes or no!')
```

Default Arguments…

What is the output of the following code? 5 or 6

```
>>> i = 5
>>> def f(arg=i):
        print(arg)
>>> i = 6
>>> f()
```

Output: 5

The default values are evaluated at the point of function definition in the *defining* scope….because def is an executable…it evaluates all the deaut args as well as constants.

Default Arguments…

What is the output of the following code?

```
>>> def f(a, L=[]):
        L.append(a)
        return L
>>> print(f(1))
>>> print(f(2))
>>> print(f(3))
```

Output:

```
[1]
[1, 2]
[1, 2, 3]
```

The default value is evaluated only once. This makes a difference when the default is a mutable object such as a list. So, accumulates the arguments passed to it on subsequent calls

Default Arguments…

If you don't want the default to be shared between subsequent calls…

```
>>> def f(a, L=None):
        if L is None:
            L = []
        L.append(a)
        return L
>>> print(f(1))
>>> print(f(2))
>>> print(f(3))
```

Output:

```
[1]
[2]
[3]
```

Keyword Arguments

- The caller identifies the **arguments by the parameter name.**
- This allows you to **skip arguments** or **place them out of order**
- **Example:**

```
>>> def parrot(voltage, state='a stiff', action='voom',
type='Norwegian Blue'):

        print("This parrot wouldn't", action, end=' ')

        print("if you put", voltage, "volts through it.")

        print("Lovely plumage, the", type)

        print("It's", state, "!")
```

Keyword Arguments…

How to call the function “parrot( )”?

```
# 1 positional argument
parrot(1000)

# 1 keyword argument
parrot(voltage=1000)

# 2 keyword arguments
parrot(voltage=1000000, action='VOOOOOM')
parrot(action='VOOOOOM', voltage=1000000)

# 3 positional arguments
parrot('a million', 'bereft of life', 'jump')

# 1 positional, 1 keyword
parrot('a thousand', state='pushing up the daisies')
```

Keyword Arguments…

Are the following function calls to "parrot( )" correct?

```
# required argument missing
parrot()

# non-keyword argument after a keyword argument
parrot(voltage=5.0, 'dead')

# duplicate value for the same argument
parrot(110, voltage=220)

# unknown keyword argument
parrot(actor='John Cleese')
```

Rules

- In a function call, **keyword arguments must follow positional arguments**.
- All the keyword arguments passed must match one of the arguments accepted by the function, and their order is not important.
- No argument may receive a value more than once.

Variable length Arguments

- Number of arguments are unknown at the time of function definition
- Need to pass more arguments than you specified while defining the function.
- These arguments are called *variable-length* arguments and are not named in the function definition
- **Example:**

```
# Function definition is here
>>> def printinfo( arg1, *vartuple ):
        "This prints a variable passed arguments"
        print "Output is: "
        print arg1
        for var in vartuple:
              print(var)
              return;
# Call printinfo function
>>>    printinfo( 10 )
>>>    printinfo( 70, 60, 50 )
```

Output:
10
70 60 50

return statement

- The statement return [expression] exits a function.
- Optionally passing back an expression to the caller.
- A return statement with no arguments is the same as return **None**.
- **Example:**

```
# Function returning None
>>> def printinfo( arg1, *vartuple ):
        "This prints a variable passed arguments"
        print(arg1)
        return  # Equal to return None

# Function returning some value
>>> def sum( x, y )
          return( x + y )
```

Local .vs. Global

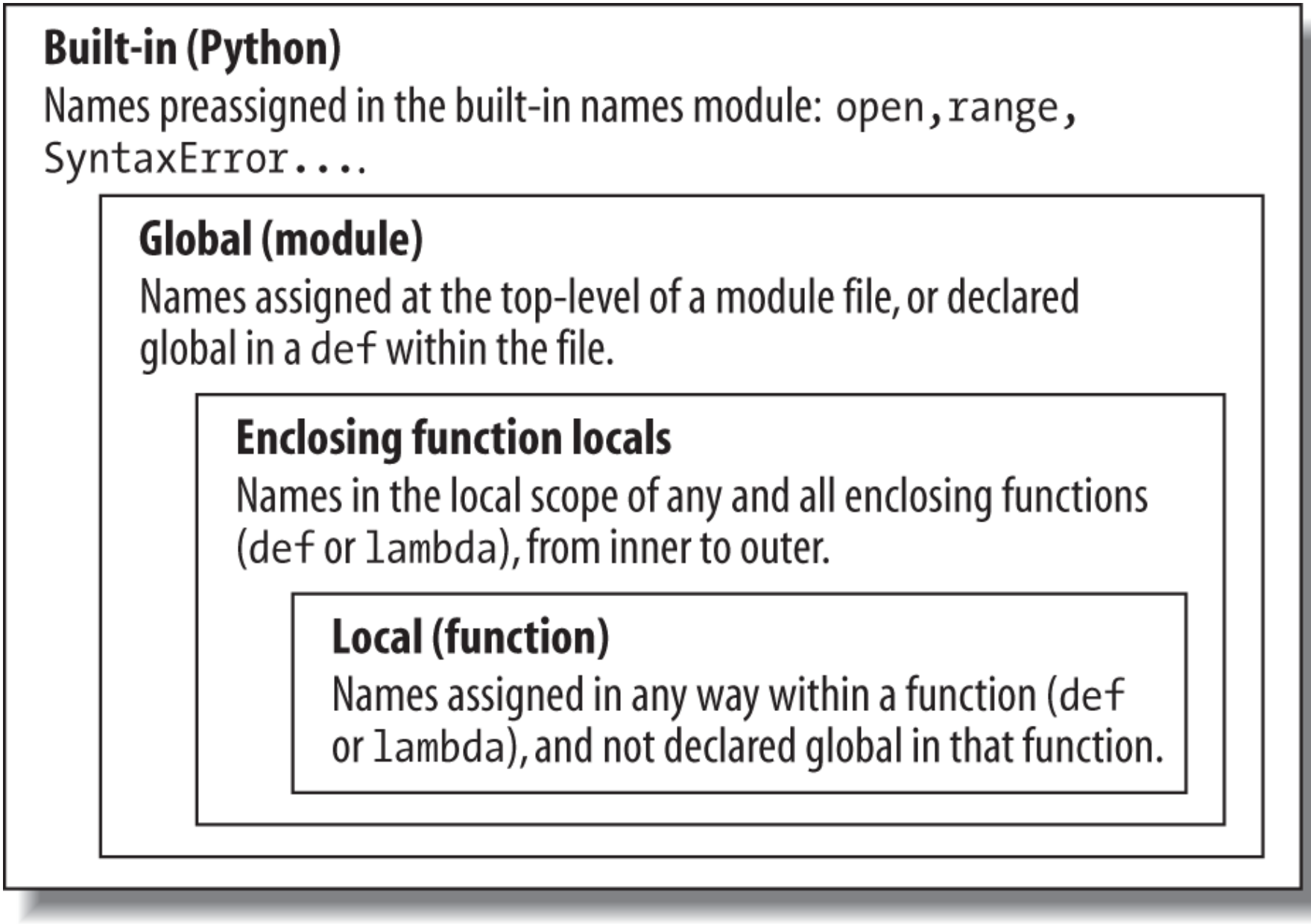
- Variables that are defined inside a function body have a local scope, and those defined outside have a global scope.
- local variables can be accessed only inside the function
- global variables can be accessed throughout the program body

Example:

```
total = 0;    # This is global variable.
>>> def sum( arg1, arg2 ):
        total = arg1 + arg2; # Here total is local variable.
        print "Inside the function sum - total = ", total
        return total;
# Now you can call sum function
>>> sum( 10, 20 );
>>> print "Outside the function global total = ", total

O/P:    Inside the function sum - total = 30
        Outside the function global total = 0
```

Local .vs. Global



Function Renaming

- A function definition introduces the function name in the current **symbol table**.
- The value of the function name has a type that is recognized by the interpreter as a user-defined function.
- This value can be assigned to another name which can then also be used as a function. This serves as a general renaming mechanism:

```
>>> def fib(n):
    """Print a Fibonacci
       series up to n."""
    a, b = 0, 1
    while a < n:
        print(a, end=' ')
        a, b = b, a+b
    print()
```

```
>>> fib
<function fib at
10042ed0>

>>> f = fib #Renaming

>>> f(100)
0 1 1 2 3 5 8 13 21 34 55
89
```

Lambda Expressions

- Small **anonymous or unnamed functions** can be created with the lambda keyword.
- They are often used as a way **to inline a function definition**
- **They are syntactically restricted to a single expression.**
- Semantically, they are just syntactic sugar for a normal function definition.

```
Syntax: lambda arguments: expression
# Function definition is here
sum = lambda x, y: x + y;

# Call sum as a function
print("Value of total : ", sum( 20, 40 ))
print("Value of total : ", sum( 30, 50 ))

Output: 60 80
```

Lambda Expressions

- **Lambda:** The name came from LISP language.
- **Syntax:**

lambdaobj = lambda *arg1, arg2,... argN : expression using arguments*

- Lambda is an expression
- Lambda body is a single expression, not a block of statements
- Lambda is used for coding simple tasks, where as functions are for bigger tasks

When to use “lambda”

- Use lambda if you want to embed a small piece of code into a statement where def cannot be embedded.
- Lambda is commonly used to create jump tables (Which are Lists or Dicts of actions)

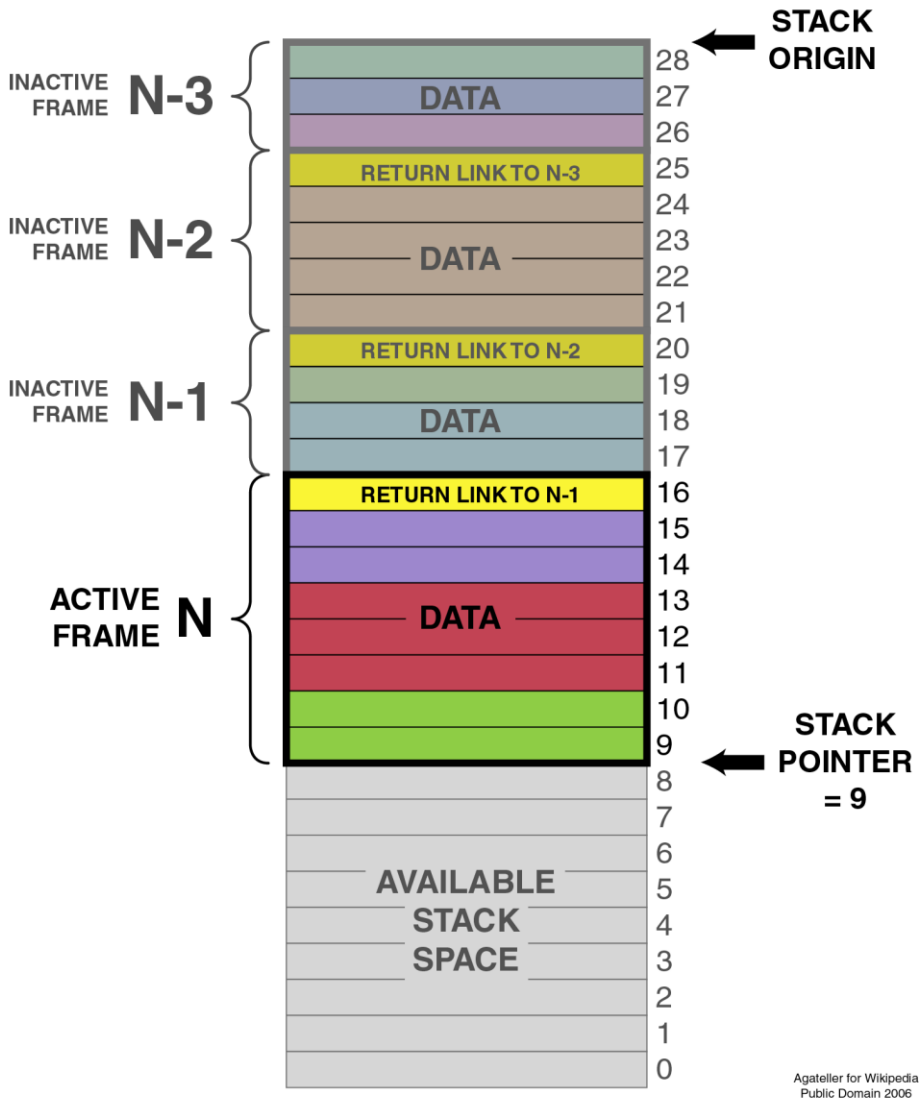
Lambda Expressions as function return val

Another Example:

```
>>>  def make_incrementor(n):
         return lambda x: x + n
...

>>> f = make_incrementor(42)
>>> f(0)
42
>>> f(1)
43
```

Visualize Function Call Stack



Visualize Function Execution Flow

Call Stack

- The call stack is a stack data structure that stores information about the current active function call
- The objects in the stack are known as "stack frames". Each frame contains the arguments passed to the function, space for local variables, and the return address
- A stack is a Last-In-First-Out (LIFO) data structure
- When a function is called, a stack frame is created for it and pushed onto the stack
- When a function returns, it is popped off the stack and control is passed to the next item in the stack. If the stack is empty, the program exits

Visualize Function Execution Flow

Call Stack

```
1  def f(a):
2      multiplier = 2
3      return a * multiplier
4
5  b = 10
6  c = f(b)
7  print(c)
```

Credits: http://pythontutor.com/csc108h.html#mode=display

Visualize Function Execution Flow

Call Stack

Python 3.3

```
1 def f(a):
2     multiplier = 2
3     return a * multiplier
4
5 b = 10
6 c = f(b)
7 print(c)
```

Frames

Global frame

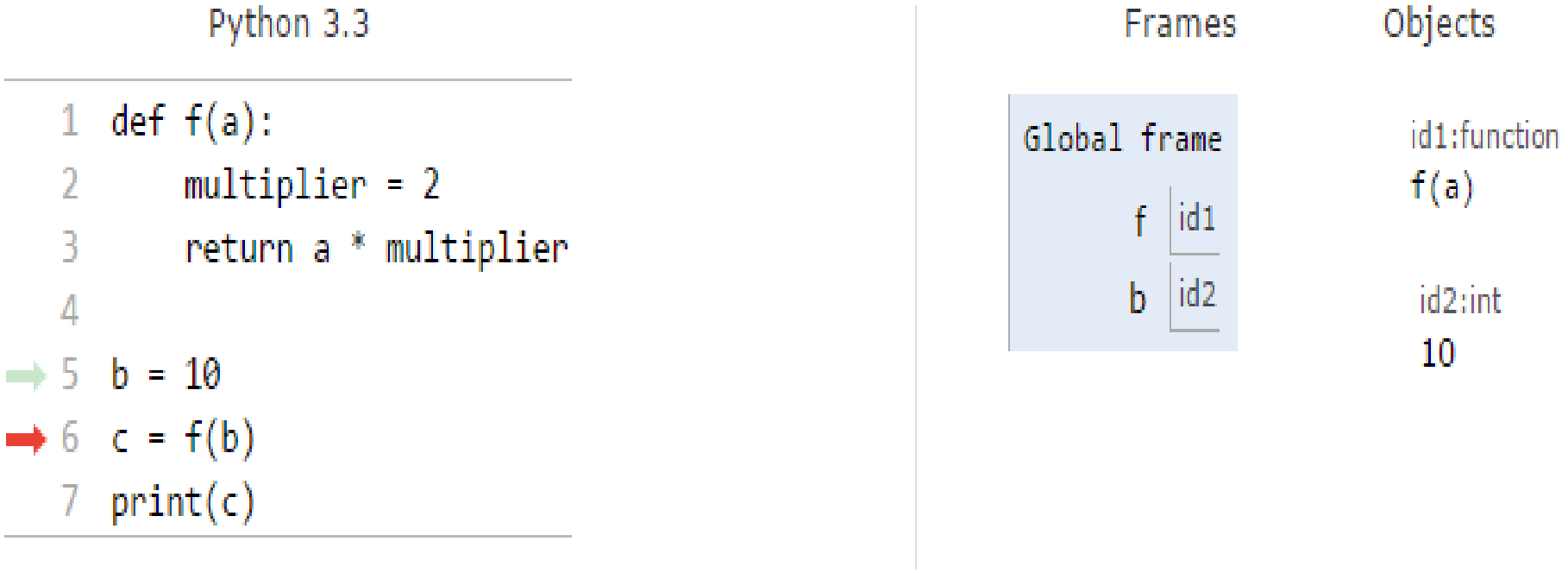
f	id1

Objects

id1:function
f(a)

Visualize Function Execution Flow

Call Stack

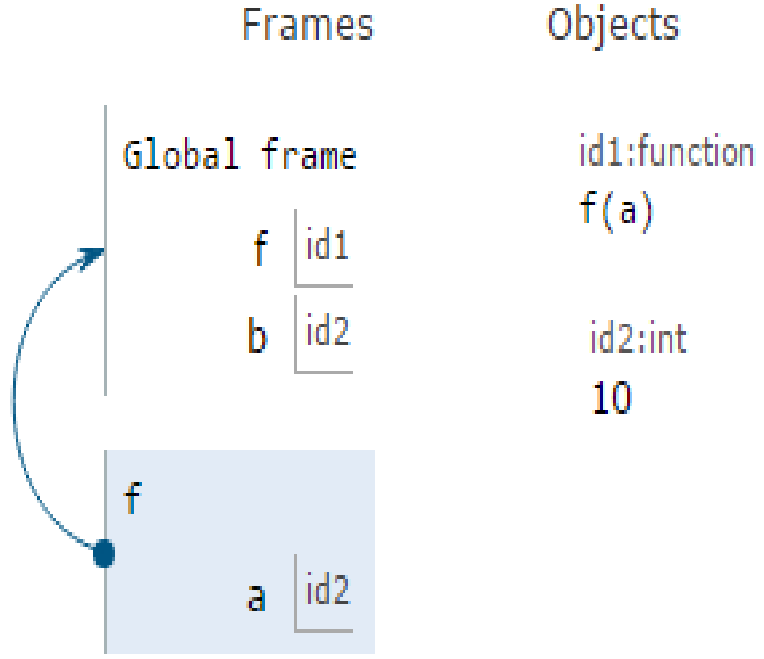


Visualize Function Execution Flow

Call Stack

Python 3.3

```
1 def f(a):
2     multiplier = 2
3     return a * multiplier
4
5 b = 10
6 c = f(b)
7 print(c)
```

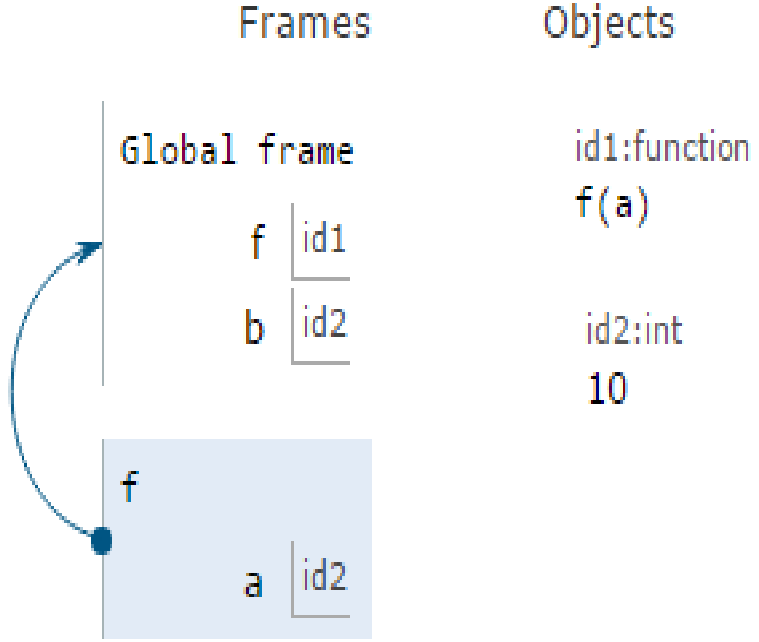


Visualize Function Execution Flow

Call Stack

Python 3.3

```
1 def f(a):
2     multiplier = 2
3     return a * multiplier
4
5 b = 10
6 c = f(b)
7 print(c)
```



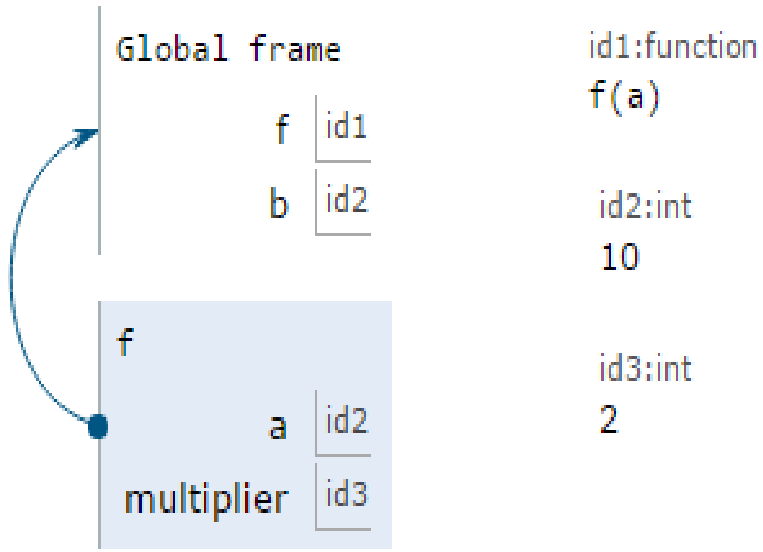
Visualize Function Execution Flow

Call Stack

Python 3.3

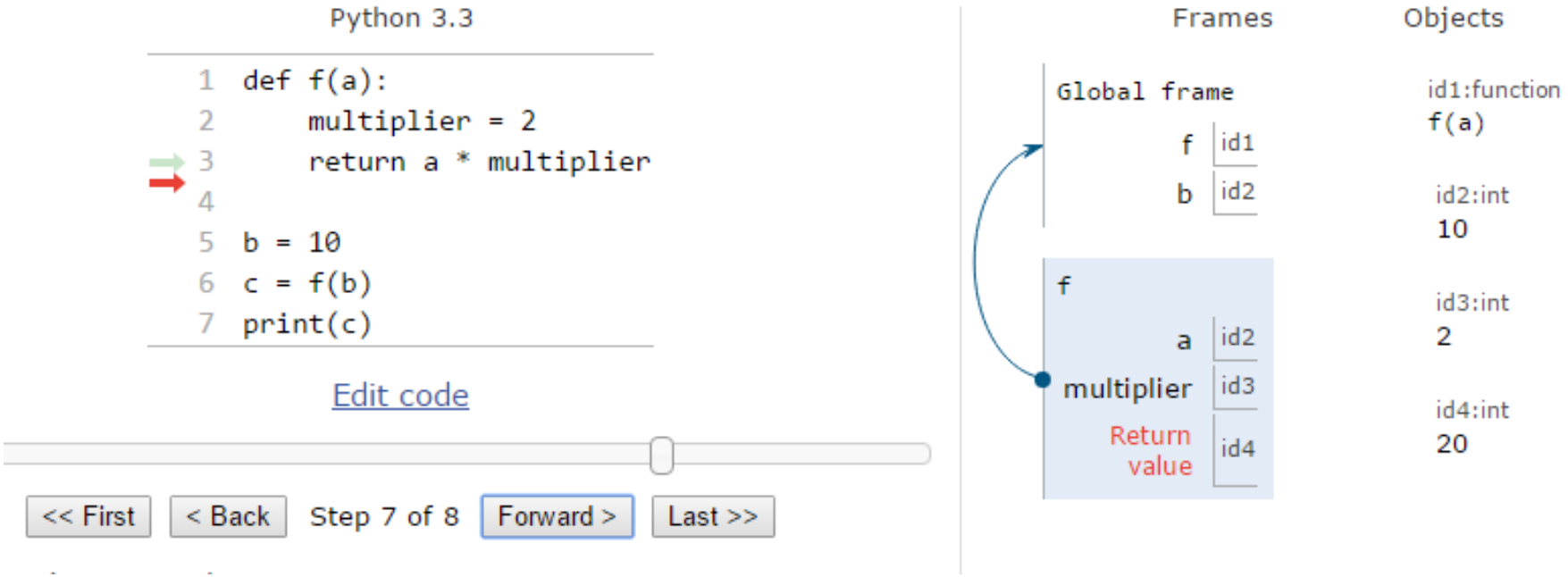
```
1  def f(a):
2      multiplier = 2
3      return a * multiplier
4
5  b = 10
6  c = f(b)
7  print(c)
```

Edit code



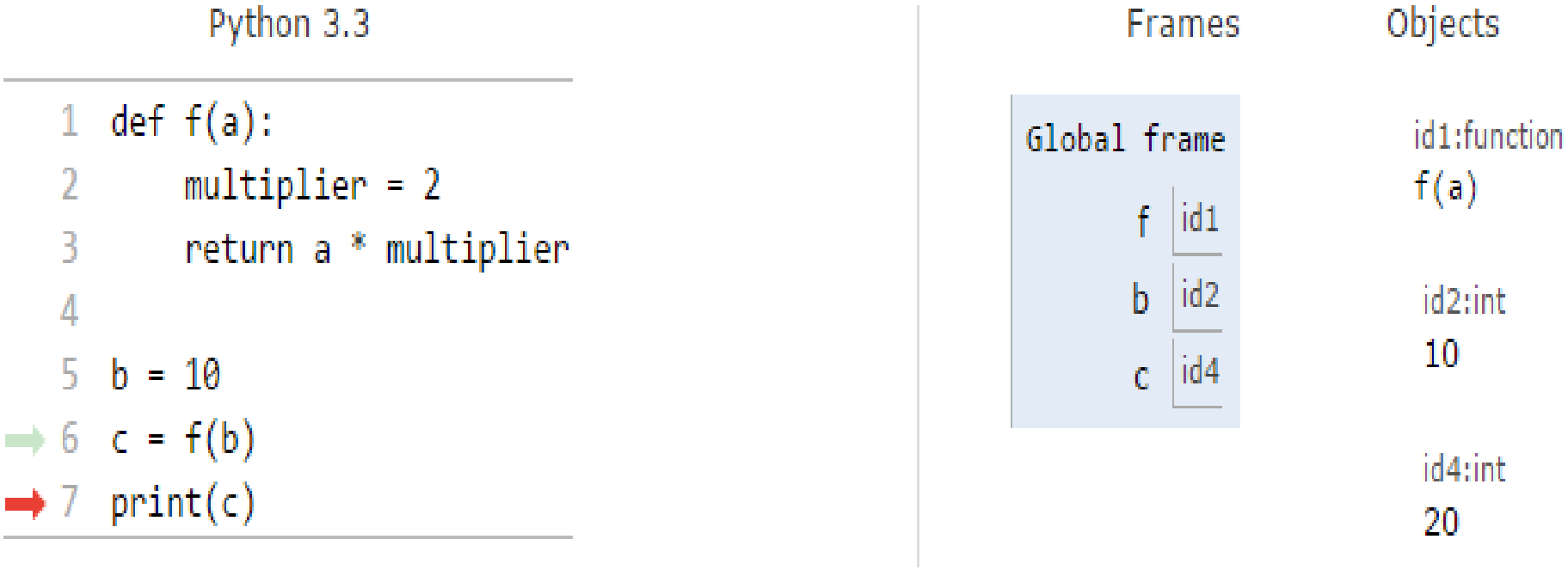
Visualize Function Execution Flow

Call Stack



Visualize Function Execution Flow

Call Stack



Class Room Exercises

1. Write a function called nested_sum that takes a nested list of integers and add up the elements from all of the nested lists.
2. Write a function called middle that takes a list and returns a new list that contains all but the first and last elements.

Summary

- ✓ Introduction to Functions
- ✓ Mutable vs Immutable Arguments
- ✓ Types of function parameters
- ✓ Scope – Local vs Global
- ✓ Lambda Expressions
- ✓ Execution flow of funcions
- ✓ Examples

Thank You